INF1009 *Object* Oriented Programming

Project Report

Project Title: *OverBaked*

Submitted by: P9 | Group 9

Jonathan Tan Jiang Ming | 2203774@sit.singaporetech.edu.sg

Goh Jun Hao Xavier | 2501315@sit.singaporetech.edu.sg

Xavier Chin Wei Le | 2500745@sit.singaporetech.edu.sg

Campion Chew Kok Sian | 2500309@sit.singaporetech.edu.sg

Lee Ruo Chen | 2500530@sit.singaporetech.edu.sg

Date of Submission: 27/3/2026

1. Introduction 3

1.1 Overview of the Engine and Improvements made since part 1. 3

1.2 Overview of OverBaked 4

1.3 The overall scope and goals of OverBaked 4

2. System Design and Implementation 5

2.1 Overall Design 5

Core modules 5

SOLID Principles 6

Entity Management Principles 7

Design Patterns 9

2.2 Key Features 10

2.3 Innovation 12

3. UML Diagram 13

4. Team Contributions 14

Introduction

1.1 Overview of the Engine and Improvements made since part 1.

Overview

The engine is created to be a custom 2D game framework built on top of the LibGDX library in Java. With modularity and a decoupled architecture in mind, our engine supports complex entity interactions, rendering, and state management. To achieve this, the architecture is split into two distinct layers, the Abstract Engine and the Logic Engine.

The Abstract Engine acts as the foundation, providing developers with robust and reusable systems. By encapsulating complex, low-level systems such as collision detection, memory management and movement logic behind clean application programming interfaces, developers can focus entirely on game design without modifying the backend engine logic.

The Logic Engine, which is what we have worked on during part 2, demonstrates how the Abstract Engine can be leveraged to create complex simulations or games. Throughout the development of the Logic Engine, architectural bottlenecks were identified in the base framework. A refactoring phase of the Abstract Engine was performed to transform it from an engine that covered some bases to a more robust and extensible framework capable of supporting a wide variety of game development.

Architectural Improvements

Single Responsibility in *Entity* Management

The previous EntityManager had evolved into a "God Class", handling updates and rendering of game and map entities simultaneously. To adhere to the Single Responsibility Principle, the manager was split into three specialized classes:

- *EntityRegistry*: Strictly handles data storage, memory, and entity lifecycles. It also categorizes entities into dedicated lists such as *updatableEntities*, *renderableEntities*, *collidableEntities*, allowing managers like *CollisionManager* to bypass iterating over the entire master list.
- *EntityRenderer*: Strictly handles the visual rendering of entities.
- *MapEntityLoader*: Strictly handles the initialization of world entities with Tiled Maps.

Interface Segregation and Components

The foundational *Entity* class was fundamentally redesigned to embrace a more component based architecture. By introducing specific interfaces to allow entities to inherit characteristics such as collision, renderable, or updatable they require. This strict adherence to the Interface Segregation Principle prevents entities from carrying unnecessary logic methods or attributes.

Similar to the collision component, the *createAnimationComponent* method allows entities to choose if they need sprite sheet slicing to have animation sprites.

Entity Segregation

To further optimize memory and structure, a *DynamicEntity* class was extended off the base *Entity* class. This ensures that static environment objects do not waste memory on velocity or movement attributes, keeping the base *Entity* class as lightweight as possible.

Abstracted _Scene_ Management

The *SceneManager* was completely decoupled from specific game logic. Previously, scenes were hardcoded into the manager itself to transition between scenes. Now, scenes are registered through the *GameMaster* and allows developers to load different scenes with the *loadScene* method. The scene manager has also been removed from the *EventObserver* role which is now given to the *SceneController* which allows more freedom of coding. A breadcrumb trail has also been implemented to allow developers to return to previous scenes if needed safely.

Enhanced Output Management

The *OutputManager* was upgraded to act as an observer. It utilizes LibGDX's Preference class to read and write persistent user settings across different sessions. It also reacts to a general event that allows audio adjustments in real time.

Viewport integration to Camera Manager

The *CameraManager* has added LibGDX's Viewport class which allows dynamic screen sizes and aspect ratios to be handled. This gives the developer more control over the Camera system.

1.2 Overview of OverBaked

OverBaked is a simulation game designed to recreate the fast-paced environment of a fast food restaurant kitchen. It addresses the challenge of helping players understand how to manage time, multitask, and maintain efficiency under pressure while preparing different types of burgers. Through gameplay, players learn the sequence of cooking steps and the importance of speed and accuracy in order fulfillment.

The target audience includes casual gamers and individuals who are curious about the experience of working in a fast food kitchen, particularly those interested in time-management and simulation-based games.

1.3 The overall scope and goals of OverBaked

The primary goal of OverBaked is to simulate the dynamic and high-pressure environment of a fast food restaurant kitchen. Within this scope, the game aims to:

- Provide players with an immersive experience that reflects the pace and workflow of a

real kitchen environment.
- Teach players the preparation and assembly of different burger recipes through interactive gameplay.
- Develop players' decision-making skills by requiring them to prioritize tasks under time constraints.
- Improve players' time management and multitasking abilities in a fast-paced setting.
- Encourage efficiency and accuracy in completing customer orders.

System Design and Implementation

2.1 Overall Design

The engine is built to be a highly modular, event-driven architecture designed with enforced SOLID principles and software architectural design pattern is divided into several independent core modules. The relationships between these modules are detailed below and visualized in the UML diagram.

Core modules

The Core of the Application

- *GameMaster*: The root entry point of the application. It acts as the global coordinator, responsible for initializing the game's lifecycle and triggering the startup sequence to activate the managers, factories, mappings, and observers properly.
- *GameRegistry* & *GameAssets*: A set of configuration helper classes that helps the *GameMaster* initialize the game assets, mapping of global inputs, and registration of entity factories.
- *GameContext*: A data container that encapsulates all global managers, passing them safely to specific managers or scenes.

Scene & Flow Management

- *SceneManager*: A global state machine that directs the active *Scene*. It utilizes a navigation stack to track scene history, allowing for dynamic routing between scenes.
- *Scene*: Specific scenes have been created which contains the logic behind what is shown on the game screen.
- *SceneController*: Handles event logic for the scenes.

Communication Event Bus

- *EventManager*: Acts as the global broadcasting system, where it notifies all event observers of events happening.

Rendering

- *EntityRenderer*: Retrieves the list of renderable objects and sorts them based on their vertical position to prevent visual layering errors.
- MapManager: Handles tiled map loading and rendering, while converting map objects into in-game entities. It also manages camera updates and resource disposal.

Supporting Managers

- *InputManager*: Captures play input and converts it into events dispatched through the *EventManager*, decoupling input handling from game logic.
- *CollisionManager*: Triggers respective collision logic for entities when collision is detected between other entities.
- MovementManager: Handles movement and basic physics for dynamic entities, updating position and velocity consistently.
- *OutputManager*: Manages persistent settings such as audio. It also acts as an event observer, allowing real time updates when settings are changed.

SOLID Principles

The engine actively prevents structural decay by meticulously adhering to the SOLID principles.

Single Responsibility Principle:

The architecture actively prevents the formation of "God Classes". The *GameMaster* is stripped of all specific game logic, leaving its single responsibility to manage the time rendering loop. It delegates asset loading entirely to *GameAssets*, and dependency wiring to *GameRegistry*.

Similarly, the previous entity manager was segregated into three specialized systems: the *EntityRegistry* manages *Entity* lifecycles and garbage collection, the *EntityRenderer* handles the projection and visual rendering of entities, and the *MapEntityLoader* is dedicated to parsing Tiled Maps to instantiate world objects.

The engine separates raw hardware input from semantic game logic. The *InputManager* is strictly responsible for polling hardware events, while the *EventManager* is strictly responsible for broadcasting what those hardware events actually mean.

Open/Close Principle

The engine follows the Open/Closed Principle most clearly through the *Ifactory* interface and the *EntityRegistry* pattern. The *EntityRegistry* class accepts any factory that implements *Ifactory* via *registerFactory*, and uses it to instantiate entities through a uniform *createEntity* function call without ever knowing the concrete entity type being produced. When the game needs a new entity, such as a *Stove* or *ChoppingStation*, the developer simply writes a new class implementing *Ifactory<Stove>* (e.g. *StoveFactory*) and registers it in *GameMaster.create()* with *entityRegistry.registerFactory(Stove.class, new StoveFactory(assetManager))*. At no point does *EntityRegistry*, *MapEntityLoader*, or any other abstract engine class need to be modified to accommodate the new type. The same principle extends to *MapEntityLoader*, which maps *TiledMap*

type strings to factories via *registerTypeAlias* where new map object types are supported purely by registering an additional alias, not by editing a switch statement or adding conditional branches inside the loader. This design keeps the abstract engine closed for modification while remaining fully open for extension through new factory implementations.

Liskov Substitution Principle

The engine adheres to the Liskov Substitution Principle through its *Layer* hierarchy and how *Scene* consumes it. The abstract *Scene* class holds a *List<Layer>* and iterates over it uniformly in *update(float)*, *render()*, and *dispose()* – it never checks the concrete type of any layer before calling these methods. The three subclasses – *BackgroundLayer*, *EntityLayer*, and *UILayer* – each override these methods with entirely different behaviour (animated sprite playback, the entity registry updates with y-sorted rendering, and *Scene2D* stage drawing respectively), yet any combination of them can be added to a scene's layer list and the loop works correctly without modification. For instance, *TutorialScene* adds a *BackgroundLayer* followed by a *UILayer*, while *GameScene* uses an *EntityLayer* and a *UILayer* – both scenes call the same inherited *super.render()* loop and each *Layer* subtype produces its correct output. No subclass violates the contract established by *Layer*: *update()* advances state, *render* draws to screen, and *dispose()* frees resources. This means any future layer type – such as a *ParticleLayer* or *DebugOverlayLayer* – could be substituted into any scene's layer list and the *Scene* base class would process it correctly without any conditional logic or awareness of the new type.

Interface Segregation Principle and Component Design

A core architectural pillar of the Abstract Engine is its strict adherence to the Interface Segregation Principle, combined with a Component Based design philosophy. The principle states that no classes should be forced to have methods it does not use. To achieve this, behaviours were segregated into highly specific interfaces instead of forcing a rigid set of behaviours onto the abstract *Entity* class.

- *IUpdatable*: Contracts the *update* method.
- *IRenderable*: Contracts the *render* and sorting method (used to *render* Entities based on depth).
- *ICollidable*: Contracts the *onCollision* method for collision interaction logic.
- *Moveable*: Contracts the *updateMovement* method for movement logic.

By segregating these behaviours, the engine allows subclasses to inherit the characteristics they require. For example:

- A static background decoration (eg. the *Counter* class) only implements *IRenderable* and *ICollidable*. It does not implement the *update* method for optimization.
- An invisible map trigger (eg. a *Door* class) only implements *ICollidable*.
- The *DynamicEntity* class explicitly implements *IUpdatable* and *Moveable*, ensuring that entities capable of movement have the needed contracts.

Furthermore, having the interface segregation, the engine's core managers are optimized to have dedicated entity lists categorized by the *EntityRegister*. When the *EntityRenderer* draws a frame,

it only iterates over objects that implement the *IRenderable* interface, rather than traversing the entire master list of entities and checking if an object happens to be renderable.

To support this principle at the data level, the *Entity* class utilizes Components. Rather than inheriting animation or collision directly, entities can request to have the components with the creation methods which allows a basic *Entity* to be lightweight upon instantiation.

Dependency Inversion Principle

The final pillar of the framework's architecture is its strict adherence to the Dependency Inversion Principle which indicates that high-level modules should not depend on low-level modules, failing to adhere results in tight coupling between systems. The Engine prevents this tight coupling through the implementation of the *GameContext* and constructor injection:

- *GameContext* Creation: Rather than allowing systems like the *AssetManager*, *SceneManager*, or *EventManager* to manage their own lifecycles as global singletons, the framework centralizes their initialization through the *GameMaster*. It creates these core systems once and bundles them into an encapsulated data container known as the *GameContext*.
- Constructor Injection: High-level classes such as a specific *Scene* or an EntityFactory, are made to retrieve their required parameters such as the managers from the *GameContext*. When the *GameRegistry* initializes the game, the *GameContext* is injected into the constructors of the high-level objects to provide a neater and encapsulated method to provide the managers.

This provides the engine to be highly modular and testable as dependencies are explicitly passed down rather than globally fetched.

Entity Management Principles

Generics for Type Safety

The abstract engine makes extensive use of generics to maintain type safety without requiring manual casting. The *Ifactory* interface is declared as *Ifactory<T extends Entity>*, which uses a bounded generic type parameter to ensure that any factory implementation can only produce objects that are subtypes of *Entity*. This means that when *EntityRegistry* calls *factory.createEntity(x, y, width, height)*, the returned object is already guaranteed to be the correct concrete type at compile time. The *EntityRegistry.createEntity* method demonstrates this further – it accepts a *Class<T>* parameter and returns *T* directly, so calling code such as *entityRegistry.createEntity(Bun.class, x, y, w, h)* receives a *Bun* reference back without any cast. The *registerFactory* method similarly uses generics to bind a *Class<T>* key to an *Ifactory<T>* value, ensuring that the factory registered for a given class always produces the correct type. This eliminates the fragile pattern of storing entities as raw *Object* references and casting them on retrieval, which would be error-prone and violate type safety.

Components and *Layers*

The engine cleanly separates its architecture into two layers. The abstract engine layer, con-

tained within the *abstractEngine* package, provides all game-agnostic infrastructure: *Entity*, *DynamicEntity*, *CollisionComponent*, *MovementComponent*, *AnimationComponent*, the *Scene* and *Layer* hierarchies, and all manager classes. The application layer, contained within the *logicEngine* package, builds on top of this foundation with game-specific concrete classes such as *Player*, *ChoppingStation*, *Stove*, *Counter*, and the various food-related entities. Within each entity, behaviour is further decomposed into components. Rather than placing all functionality into a single monolithic class, the engine delegates collision detection state to *CollisionComponent*, physics and movement data to *MovementComponent*, and sprite animation to *AnimationComponent*. A *DynamicEntity* composes these components together – for example, it holds a *MovementComponent* that tracks velocity, acceleration, and directional input state, while the *Entity* base class holds a *CollisionComponent* for spatial bounds. The rendering layer is also composed through the *Layer* abstract class, where *BackgroundLayer* handles animated tile backgrounds, *EntityLayer* manages entity *update* and *render* cycles, and *UILayer* wraps a *Scene2D* Stage for interface elements. Each *Scene* holds a *List<Layer>* and iterates over it uniformly, meaning layers can be mixed and matched per scene without any conditional logic.

Object (*Entity*) Management

The engine implements entity management through manager classes that oversee lists of common objects and invoke updates polymorphically. *EntityRegistry* maintains separate lists for different concerns: a master List<*Entity*> for all entities, a List<*IUpdatable*> for entities requiring per-frame updates, a List<*IRenderable*> for entities that need to be drawn, and a List<*Entity*> for collidable entities. When an entity is added via *entityRegistry.add(entity)*, the registry inspects which interfaces it implements and automatically categorises it into the appropriate lists. During the game loop, entityRegistry.*update*(deltaTime) iterates over only the *IUpdatable* list and calls *update* on each entry, while *EntityRenderer.render* iterates over the *IRenderable* list, sorts by Y position, and calls *render* on each. This is coding to interfaces in practice – the manager does not know or care whether the object is a *Player*, a *Stove*, or a *Plate*; it simply invokes the interface method and each entity handles its own behaviour through polymorphism and overriding. The *CollisionManager* demonstrates the same principle: it receives a list of collidable entities, inserts them into a *SpatialHashGrid* for efficient spatial lookup, checks for overlapping bounds, and when a collision is detected, calls *onCollision* on any entity that implements *ICollidable*. The *Player* responds to collision by reverting to its previous position, while a *Counter* simply blocks movement – each entity handles its own collision response. Entities are assigned to the manager on creation and can be dynamically removed through *markForRemoval*, which queues entities for safe removal at the end of the *update* cycle via *processRemovals*, preventing concurrent modification issues when entities are destroyed mid-frame. This design ensures that the manager classes coordinate and delegate without doing the work themselves, while the entities and their components are responsible for their own behaviour.

Design Patterns

Creational Patterns

Factory Pattern

The engine applies the Factory Pattern through its *Ifactory* interface to hide the construction details of individual entities. Each concrete factory – such as *BunFactory*, *StoveFactory*, or *PlayerFactory* – implements *Ifactory* and encapsulates the specific constructor logic, texture loading, and parameter wiring needed to produce that entity type. For example, *PlayerFactory.createEntity* internally fetches five separate sprite sheet textures from the *AssetManager* and passes them into the *Player* constructor, but from the caller's perspective, the creation is a single method call with just position and size parameters. The calling code never needs to know which textures are required, how the *AnimationComponent* is initialised, or what collision offsets are applied. This separation means that the construction of a single object is hidden behind the factory, and any changes to how a *Player* is constructed – such as adding a new animation state – only require modifying *PlayerFactory*, not every location where a player is created.

Abstract Factory Pattern (Factory Registry)

The engine goes beyond individual factories by implementing a Factory Registry pattern through *EntityRegistry*, which acts as a centralized registry that groups multiple individual factories under a common interface. In *GameMaster*.create, each factory is registered with its corresponding class type – for instance entityRegistry.*registerFactory*(*Bun*.class, new *BunFactory*(assetManager)) and *entityRegistry.registerFactory(Stove.class, new StoveFactory(assetManager)).* Once registered, any part of the codebase can request entity creation through the registry using *entityRegistry.createEntity(Bun.class, x, y, w, h)* without knowing which concrete factory handles the creation. The *MapEntityLoader* extends this further with *registerTypeAlias*, which maps string-based type names from *TiledMap* data to factories, allowing the engine to create entire families of related game entities from map files without any concrete class references in the loader itself. This approach keeps the code loosely coupled, because the systems that consume entities – such as *InteractionManager* or *CollisionManager* – never need direct associations with the constructors of the entities they work with. Adding a new entity type to the game only requires writing a new factory class and registering it, without modifying any existing creation or management code, which satisfies both the Single Responsibility Principle and the Open/Closed Principle.

Structural Patterns

Facade Pattern

The *GameMaster* acts as the Lifecycle Facade for the external systems which is the LibGDX Launcher. To hide the intricate subsystems and logic, it implements simple methods for the launcher to use.

Initializing a complex game engine requires a strict order of operations. The *GameRegistry* class acts as a structural facade pattern by providing a single registerAll method, it hides the complexity of the engine initialization (factory registration, input mapping, asset binding) from the *GameMaster*.

Flyweight Pattern

To ensure high performance and low memory consumption, the engine utilizes the Flyweight Pattern by incorporating the *AssetManager* (The Flyweight). When the MapManager loads a map such as the restaurant level which consists of thousands of floor and wall tiles, it does not create thousands of individual texture entities. The *AssetManager* loads the *TiledMap* into the memory once and the instances hold a reference pointer back to that single shared *TiledMap* texture. Other entities that require textures or even sounds and music get their assets from the *AssetManager*, hence saving a lot of memory.

Behavioural Patterns

Observer Pattern

The *EventManager* acts as our broadcaster, announcing semantic events (Event.GameStart or Event.PlayerInteract). Systems such as *OutputManager* can choose to be an *EventObserver* to react to game states and events and perform their onNotify methods accordingly. This achieves loose coupling through an event-driven communication bus.

2.2 Key Features

Core Engine Architecture

Manager-Based System

- *EventManager*: Centralized event publishing/subscription system
- *InputManager*: Handles player input and maps it to events
- *SceneManager*: Manages scene loading, updates, and rendering
- *EntityRegistry*: Tracks all active entities in the world
- *EntityRenderer*: Renders entities in proper order
- MovementManager: Handles physics and entity movement
- *OutputManager*: Manages game settings and configuration

Entity System

- *Entity* Registry: Decoupled entity management
- Factory Pattern: *Ifactory<T>* for flexible entity creation
- Dynamic vs Static Entities: *DynamicEntity* for moving objects, base *Entity* for static
- Collision System: *ICollidable* interface for collision detection

- Rendering: *IRenderable* and *IUpdatable* interfaces for flexible behavior

Map/Level Loading

- *MapEntityLoader*: Loads entities from map definitions
- *Scene*-based architecture: Multiple scenes (Main Menu, Game *Scene*, etc.)

Game Logic Implementation

Cooking Game Mechanics

Food Order → *Player* → Station Interactions → Ingredient Preparation → Recipe Matching

- *ChoppingStation*: Raw ingredients → Chopped (Lettuce, Tomato, Cheese)
- *Stove*: Raw Patties → Cooked/Burnt (with cooking timer)
- *Counter*: Holds stacked cooked ingredients
- *Plate*: Assembles ingredients and matches against recipes

Food State System

- *FoodState.Raw* → *FoodState.Cooked* → *FoodState.Overcooked*
- Ingredient tracking with textures for each state.
- Order-independent recipe matching via *BurgerRecipe.fromIngredients()*.

Recipe System

3 Burger Recipes with ingredient frequency maps:

- Classic Burger (*Bun*, Lettuce, Tomato, Cheese, Patty)
- Double Patty (*Bun*, Tomato, Cheese, 2x Patty)
- Green Double (*Bun*, Lettuce, Cheese, 2x Patty)

Flexible matching: Order-independent ingredient validation

***Player* Mechanics**

- Single-item holding: Can carry one Ingredient or *Plate* at a time
- Interaction range: 80-unit radius for picking up/using stations
- Animation states: Idle, Running, Chopping
- Collision avoidance: Can't move through stations/walls

Asset & Configuration Management

Asset Loading

- *GameAssets*: Centralized texture/audio loading
- *AssetManager*: LibGDX asset pooling and management
- Lazy loading: Assets loaded on demand

Game Context

- Dependency Injection: '*GameContext*' passes all managers to subsystems
- Decoupling: Loose coupling between systems via events and context

Game Registry

- Factory registration: Maps entity types to their factories
- Input mappings: Keyboard/controller bindings
- Audio setup: Event-driven sound playback
- Observer pattern: Event listeners for game events

Gameplay Flow

1. Food order comes in
2. *Player* picks up raw ingredient from ingredient box
3. Place ingredient on appropriate station (Chopping/*Stove*)
4. Prepare the ingredient (chop/cook)
5. Picks up cooked ingredient from station
6. Place ingredients on *Plate*
7. Repeat step 2-6 until plate has all the ingredient to match the food order
8. If matched → *Plate* is locked and burger is complete
9. Submit plate → Score points

2.3 Innovation

OverBaked is an engaging and interesting game as it focuses on developing core cognitive and practical skills through fast-paced, interactive gameplay. Rather than being purely entertainment-driven, the game integrates skill-building elements that challenge the player in meaningful ways.

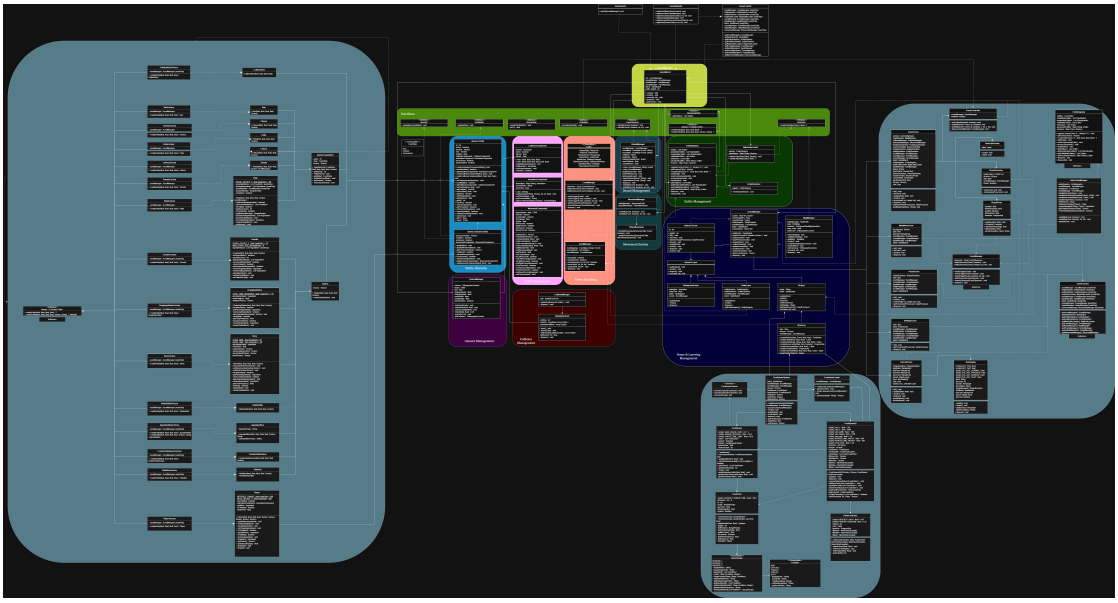
One key aspect is pattern recognition. As players repeatedly prepare specific types of burgers, they begin to recognise recurring sequences of actions. This allows them to develop mental patterns and muscle memory, enabling faster and more efficient gameplay over time. The learning curve rewards practice, making progression feel natural and satisfying.

Another important element is decision-making and time management. Each customer order comes with a time constraint, requiring players to think critically about task prioritisation. *Player*s must decide which orders to complete first, how to sequence their actions, and how to balance speed with accuracy. This creates a dynamic environment where poor decisions can lead to delays, while effective planning leads to success.

Overall, the combination of pattern recognition, time pressure, and task prioritisation makes OverBaked both challenging and rewarding, offering an experience that is not only fun but also mentally stimulating.

UML Diagram

Link to UML Diagram



Team Contributions

AI Teammate

- Report refinement

Xavier Goh

- Interactive KeyDisplay
- Game *Scene*
- Main Menu
- Pause *Scene*
- Results *Scene*
- Timer Bar
- Tutorial *Scene*
- Bug fixes
- UML diagram overview

Xavier Chin

- FoodQueueSystem

- FoodQueueUI
- FoodOrder
- FoodAssetLoader
- FoodQueueListener
- OrderCardEntry
- BurgerRecipe
- FoodItem

Jonathan

- *Entity* Design
- *Entity* Factories
- *Entity* Interactions
- Bug fixes

Campion

- Interaction/Chopping
  - *InteractionManager*
- EntityManager Refactoring
  - *EntityRegistry*
  - *EntityRenderer*
  - *MapEntityLoader*
- *EventManager* Refactoring
  - *InputManager*
  - *EventManager*
- *Ifactory* Refactoring
- Bug fixes

Ruo Chen

- *Entity* and *DynamicEntity* Refactoring
  - Interface Segregation and Specialized Interface Lists
- *OutputManager* Refactoring
- Animation Component
- *Scene* Refactoring
  - *SceneManager* overhaul
- *GameMaster* Refactoring
  - *GameContext*
  - *GameAssets*
  - *GameRegistry*
- *Scene*/interaction bug fixes
- Map, Sprite, Sound, Music design